第8讲：大模型的有监督微调

杨沐昀

哈尔滨工业大学计算学部
语言技术研究中心

yangmuyun@hit.edu.cn

有监督微调(SFT)

- 也被称为指令微调（Instruction Tuning）
- 在大规模预训练语言模型的基础上，利用带标注的特定任务数据进一步训练模型，以提升其遵循指令和解决实际问题的能力

主要内容

- 提示学习与上下文学习
- 有监督精调的基本原理
- 指令数据构建
- 微调策略

提示学习与上下文学习

- 大模型的基础能力
- 特点1：无需微调模型权重
- 特点2：直接调整输入形式引导模型完成任务
- 局限性：
  - 提示的设计通常依赖人工调试，无法保证通用性
  - 在复杂任务上的作用有限

提示学习Prompt Learning

- 又被称为提示工程（Prompt Engineering）
- 传统监督：依赖大量有标注数据来训练模型
- 提示学习：设计和定义一个特定的“提示”函数，引导模型生成符合给定任务要求的响应
  - 调整输入的表达方式，使得大语言模型能够在仅有少量标注数据或甚至没有标注数据的情况下进行有效的任务适应
  - 从而实现**小样本学习**（Few-shot Learning）或**零样本学习**（Zero-shot Learning）

提示学习诞生于预训练模型时代

- **预训练：**最小化输出文本的困惑度，该阶段使用了海量的互联网数据来训练模型，使得大语言模型具备了丰富的“世界知识”
- **精调：**是针对特定任务进行优化，以提升预训练语言模型在下游任务上的表现。下游任务不同，精调的训练目标也不同，多种多样的下游任务决定了千变万化的精调目标

提示学习考量——技术角度

- 预训练阶段的任务始终都是稳定的
  - 预测下一个词、掩码填空……
- 为了使下游任务从形式上更贴近预训练任务，从而更好地利用模型在预训练阶段获取的知识
  - 构建与预训练阶段相似的输入提示
  - 早期提示学习或提示工程诞生的重要启蒙思想之一

提示学习考量——哲学角度

- 人类学习过海量的基础知识之后，在面临特定任务时，仅需要极少代价的适应性学习即可应对同类型任务

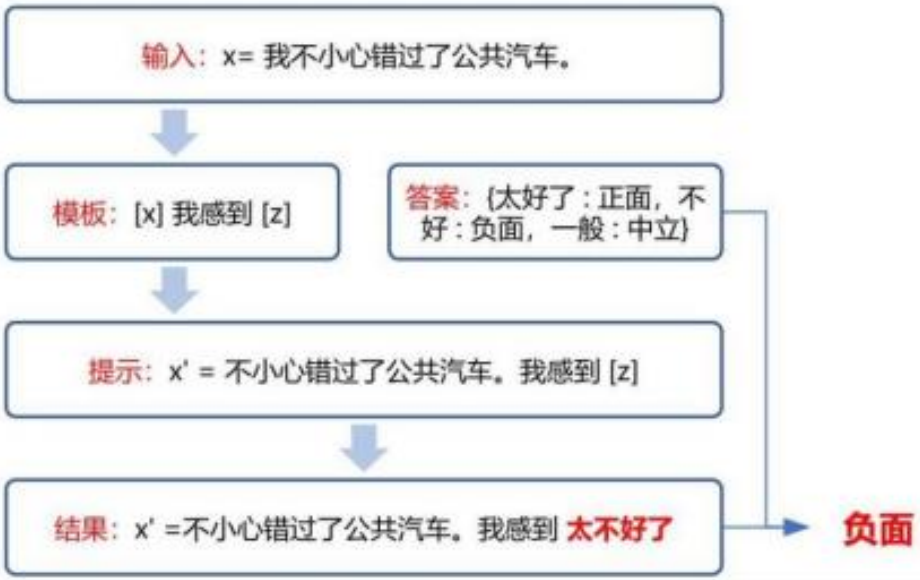
- BERT-Base为例，其训练语料高达10B 词元，而一个普通人从出生到高中为止接触的学习资料也就10B 词元左右

提示学习考量——成本角度

- 当研究者初步探索将提示学习应用于大语言模型时，虽取得一定成效，但仍显著落后于经过精调的大语言模型
- 同一时期，OpenAI发布了大语言模型产品ChatGPT。从相关技术报告来看，ChatGPT的训练语料达500B 词元，这使得其拥有更为丰富的“世界知识”
- 大语言模型庞大的参数量也让面向下游任务的精调变得极为困难

提示学习的流程

- 使用提示学习完成预测任务的流程主要包含三个阶段：
  - 提示添加
  - 答案搜索
  - 答案映射

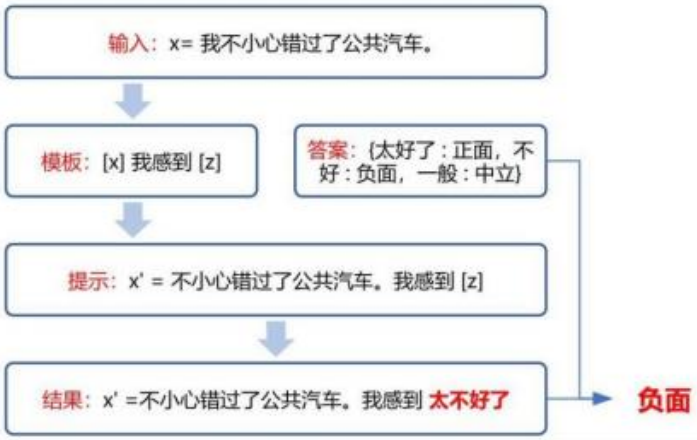


提示学习的流程——提示添加

- **提示添加**: 这一阶段的关键是设计一个有效的模板，将原始输入x转换成带有未填充槽（slot）的文本提示x'
- 模板的设计直接影响到任务的完成效果。不同的提示可能会显著改变模型的推理路径和输出结果，因此提示设计被视为提示学习中的一项关键技术。

提示学习的流程——提示添加

- **以情感分析任务为例,设计模板如下:**
  - [X] 我感到[Z]
  - 其中，[X]插槽中填入待分类的原始句子，[Z]插槽中为需要语言模型生成的答案



提示学习的流程——提示添加

- **以情感分析任务为例,设计模板如下:**
  - [X] 我感到[Z]
  - 假如原始文本x =“我不小心错过了公共汽车”
- 通过模板，整段提示将被拼接成:
  - x' =“我不小心错过了公共汽车。我感到[Z]”。

提示学习的流程——答案搜索

- 将构建好的提示整体输入语言模型后，需要找出语言模型对 [Z]处预测得分最高的文本$\hat{z}$
- 在简单的生成任务中，答案可以是文本空间中的任意内容，而在一些分类任务中，答案是一些限定的词语，例如：

$$Z = \{\text{“太好了”}, \text{“好”}, \text{“一般”}, \text{“不好”}, \text{“糟糕”}\}, 、$$

- 理想状态：答案被模型直接生成

提示学习的流程——答案搜索

- 通用搜索过程
  - 输入提示x而模型输出为z的过程记录为函数 $f_{\{fill\}}$
  - 对于每个答案空间中的候选答案，分别计算模型输出它的概率
  - 从而找到模型对 [Z] 插槽预测得分最高的输出

$$\hat{z} = \text{search}_{z \in Z} P\left(f_{\text{fill}}(x', z); \theta\right)$$

提示学习的流程——答案映射

- 但实际任务中的答案并非任意的词，而是类似“正面”、“负面”、“中性”等标签词，因此需要将模型的输入映射到任务的标签上
  - 以情感分析任务为例，输出的词将映射为如下的标签：

$$\begin{cases} \text{if } \hat{z} \in \{\text{“太好了”}, \text{“好”}\} & \hat{y} = \text{“正面”} \\ \text{if } \hat{z} \in \{\text{“不好”}, \text{“糟糕”}\} & \hat{y} = \text{“负面”} \\ \text{if } \hat{z} \in \{\text{“一般”}\} & \hat{y} = \text{“中立”} \end{cases}$$

提示学习——软提示

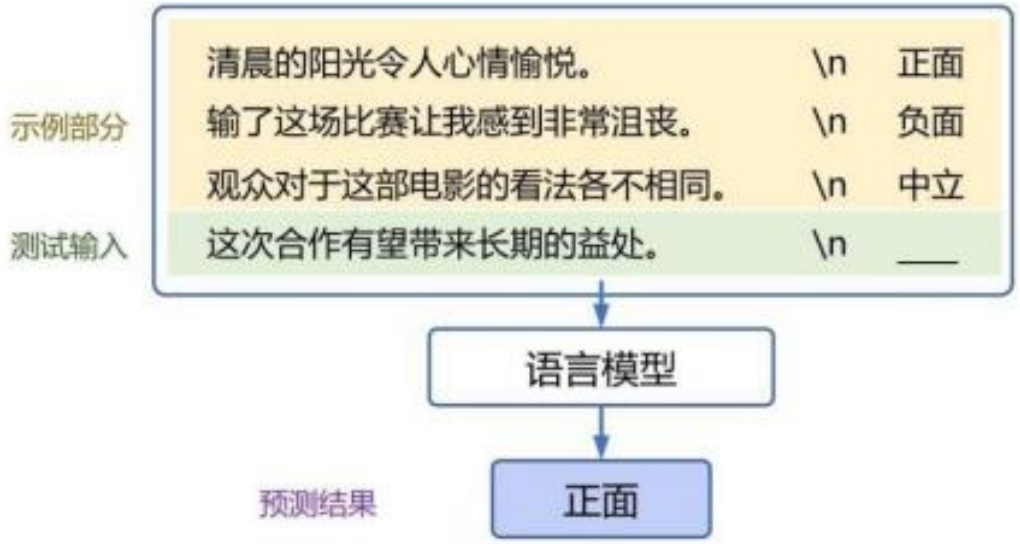
- 提示构建的目的是找到一种方法，从而使语言模型有效地执行任务
- 软提示：其直接在模型的嵌入空间中执行提示
  - 放松了模板词的嵌入是自然语言词嵌入的约束
  - 模板不再受语言模型自身参数的限制
  - 模板有自己的参数，可以根据下游任务的训练数据进行调整

上下文学习ICL

- 上下文学习（In context Learning，ICL），也称语境学习，是一种重要的学习范式
  - 最早随着 GPT-3 的诞生
- 通过输入任务的具体示例（Demonstration，通常是任务的输入和对应的输出对），推断出针对新输入的回答。
- 这种方法强调的是，模型在推理过程中，可以直接从给定的任务上下文中学习并运用新的能力（即学即用）
  - 不必经过模型训练和参数更新

上下文学习的流程

- 首先通过提供少量示例，模型学习任务的要求和模式
- 模型处理新的测试样例时，通过类比推理来生成答案



上下文学习的核心思想

- **从类比中学习**
  - 通过观察输入和示例之间的关系，推断出新输入的正确输出
  - 无需对模型进行任何参数更新，仅依赖于模型现有的参数
- **与提示学习的关系**
  - 可以看作是提示学习的一个子类
  - 上下文学习不仅包括了任务的描述，还包括了足够的示例，这些示例为模型提供了关于如何执行任务的具体指导

上下文学习面临的挑战

- 示例的质量和选择
  - 上下文学习的性能不仅依赖于输入和输出标签的准确性，还与**示例的选择（格式）**、**任务的契合度（相关性）**密切相关
- 对任务设置的敏感性
- 跨任务泛化能力
- ……

上下文学习面临的挑战

- 示例的质量和选择
- 对任务设置的敏感性
  - 上下文学习的效果对具体设置非常敏感。即使是微小的变化也可能导致性能的显著波动
    - 例如：输入示例顺序或模板变化
- 跨任务泛化能力
- ……

上下文学习面临的挑战

- 示例的质量和选择
- 对任务设置的敏感性
- 跨任务泛化能力
  - 对于一些高度复杂的推理任务，单一的上下文示例可能不足以引导模型进行高效推理
    - 判断依据复杂、推理过程繁复、需要领域知识
- ……

主要内容

- 提示学习与上下文学习
- 有监督精调的基本原理
- 指令数据构建
- 微调策略

有监督精调的基本原理

- 有监督精调（Supervised Fine-Tuning）是将预训练好的大语言模型应用到具体任务的重要步骤
- 基本原理是利用**预先标注好的监督数据**，通过优化模型权重，使其更适应**特定任务**的输入和输出需求
- 训练目标是通过最小化监督数据上的损失函数，更新预训练模型的权重
- 与传统的监督学习一样：给定一个**标注数据**集，降低模型在该标注数据集上的损失

有监督精调的形式化描述

- 设我们有一个预训练好的大语言模型f_θ，其参数为 θ
- 给定一个标注数据集$D = \{(x_i, y_i)\}_{i=1}^{N}$，其中$x_i$是指令（或输入），$y_i$是模型期望生成的响应（或输出）
- 训练的目标是最小化以下损失函数：

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} Loss(f_\theta(x_i), y_i)$$

- Loss通常选择交叉熵损失函数，用于衡量模型生成的输出分布和目标输出之间的差异

指令数据形式及其作用

- 有监督精调的关键在于构建高质量的指令-响应数据集
- 通常形式是：
  - **指令**（Instruction）：明确描述任务需求的输入，例如一个问题、命令或任务描述
  - **响应**（Response）：与指令对应的输出，通常是高质量的人类标注答案

指令数据形式及其作用

- 例子：
  - **指令**：“请解释以下 Python 代码的功能：*print([i for i in range(10) if i % 2 == 0])*。”
  - **响应**：“这段代码打印了一个列表，包含从 0 到 9 中的所有偶数。
- 与传统监督训练的标注数据对比：
  - 输入指令中不仅包含有问题，还有描述任务需求的部分
  - 模型预测从P(y|x) 变成了P(y|x,任务描述)

迁移学习视角

- 从广泛的语言预训练到特定任务优化的**知识迁移**过程
  - **预训练阶段的知识积累**：模型通过海量的无监督数据（如网页、书籍）学习了语言的结构、语义关系、上下文依赖等广泛的语言知识
  - **特定任务微调**：通过用特定数据进一步优化模型权重，从而将通用知识迁移并适配到特定的场景中

灾难性遗忘

- 灾难性遗忘是指 **模型在学习新任务时，往往会严重丧失在旧任务上的性能**
- 预训练任务和微调任务的参数共享，微调可能会影响原有的参数
- 缓解方案：
  - 正则化
  - 重放方法（Replay）
  - ……

主要内容

- 提示学习与上下文学习
- 有监督精调的基本原理
- 指令数据构建
- 微调策略

指令数据的构建

- 指令数据的质量会直接影响到有监督精调的最终效果
- 常见的方法包括：
  - 人工标注高质量数据
  - 自动生成初始数据，并通过人类审核改进

手动构建指令

- 从网上收集大量的问答数据，再通过人工筛选和过滤，或直接由标注人员手动编写提示和相应的回答
  - 耗费人力
  - 能对数据质量进行把控
- 数据集的好坏有两个重要的评价维度：
  - 质量
  - 多样性
- 高质量且多样性丰富的指令数据能够实现“以少胜多”（Less is More）

手动构建的流程：LIMA

- **LIMA**是一个65B参数的LLaMa语言模型，在没有任何强化学习或人类偏好建模的情况下，只对1000个精心策划的prompts和responses进行了标准监督损失的微调
- LIMA 实验结果表明，指令数据的质量和多样性是影响有监督精调过程的关键因素。LIMA 仅使用了 1000 条这样的高质量指令数据，便能够与那些使用数倍量指令数据的同等规模模型相媲美，甚至超越其性能。

手动构建的流程：LIMA

- **数据基础来源**
  - 高质量的网络问答社区
    - 涵盖了多个子版块，涉及不同主题
  - Super-Natural Instructions 数据集
    - 包括多种生成式任务
  - 大量标注人员手动编写的提示和回答

手动构建的流程：LIMA

- **质量保证**
  - 数据来源的可靠性
  - 人工编写数据保证质量的一致性
  - 制定了过滤规则
    - 例如，在采样社区指令数据时，选择排名较高的优质回答，将所有回答统一调整为 AI 助手的风格，删除过长或过短的回答，去除以第一人称开头的回答，并剔除包含链接的回答

手动构建的流程：LIMA

- LIMA数据样例：右图
- 手动构建数据问题
  - 需要大量的人力投入
  - 在模型需要的数据量越来越大的背景下，对于多数研究者而言，手工构建的代价难以接受

Prompt:

Tell me an interesting fact about geography

Response:

Here are some interesting facts about geography:

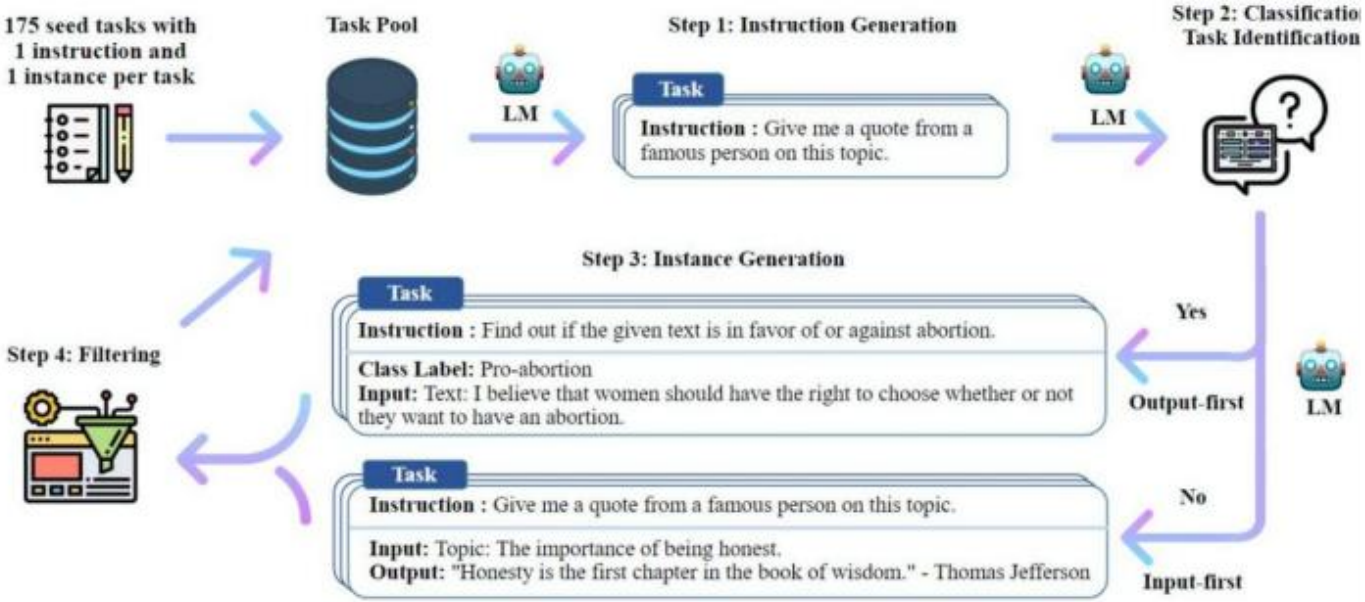
Bhutan is one of the most intriguing places in the world. The Buddhist kingdom is the only carbon-negative country on Earth, with strict environmental laws and cultural norms that preserve its stunning natural landscape. For similar reasons, Bhutan was one of the last countries to introduce television in 1999， and has no traffic lights to this day.

There are currently few territories on Earth that are unclaimed by any sovereign state (terra nullius). One example is Bir Tawil，along the border between Egypt and Sudan，which was created by a discrepancy between borders drawn in 1899 and 1902. The territory remains unclaimed by either country，because it would entail forfeiting the much larger Halaib Triangle，which is claimed by both countries. Bir Tawil has no settled population，but the land is used by Bedouins who roam the area.

The antipode of any spot on Earth [...]

自动生成指令

- 自动生成的方法主要来自于大语言模型越来越强的生成能力，利用大语言模型自动构建指令进行训练
- 代表性的工作如 Self-Instruct



Self-Instruct:生成任务指令

- Self-Instruct的第一步是生成任务指令
  - 首先，生成包含 175 个任务的小型指令数据集，称为种子指令集，用于初始化指令池
  - 接着，模型以自举（Bootstrapping）的方式，利用种子指令集生成新的任务指令：每次从指令池中随机选择 8 条任务指令，其中 6 条来自人工编写的种子指令，2 条由模型通过迭代生成
  - 将这些指令拼接为上下文示例，并用这些示例引导大语言模型生成更多新的任务指令

Self-Instruct:生成任务指令

- 生成指令时使用的prompt模版如下所示：
  - 根据下面的任务编写一系列新的任务：

任务 1：{现有任务 1 的说明}

任务 2：{现有任务 2 的说明}

任务 3：{现有任务 3 的说明}

任务 4：{现有任务 4 的说明}

任务 5：{现有任务 5 的说明}

任务 6：{现有任务 6 的说明}

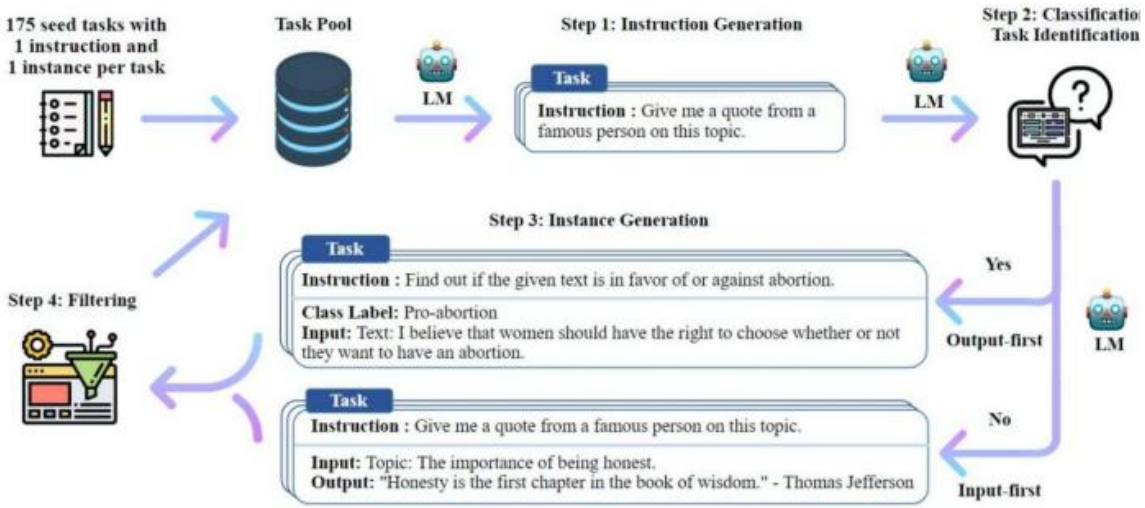
任务 7：{现有任务 7 的说明}

任务 8：{现有任务 8 的说明}

任务 9：

Self-Instruct:确定指令是否代表分类任务

- 由于后续对于分类任务和非分类任务有两种不同的处理方法，在本步骤需要对指令是否为分类任务进行判断
- 分类任务与非分类任务使用的prompt模版是不同的

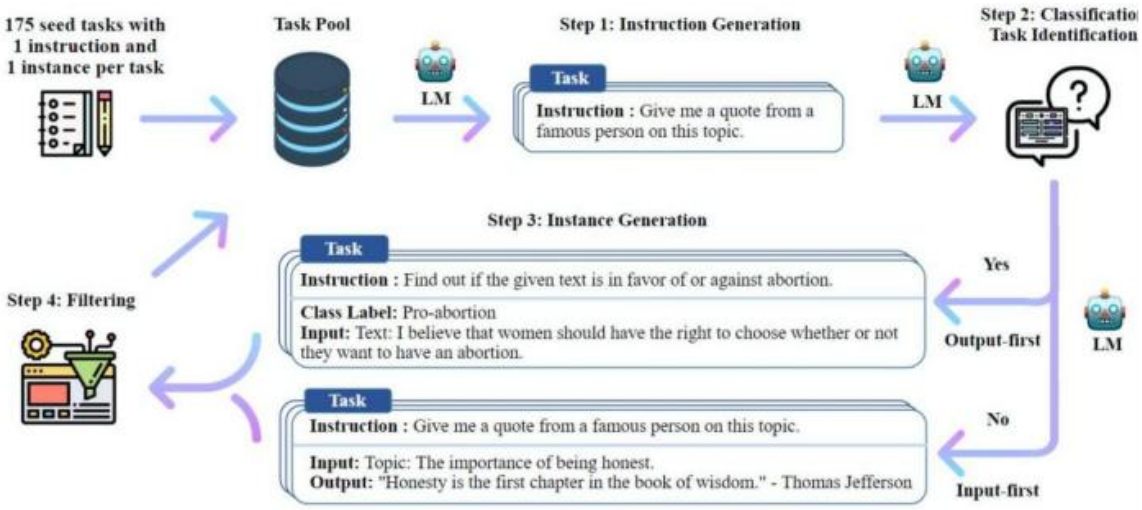


Self-Instruct:确定指令是否代表分类任务

- 判断指令是否属于分类任务：
  - 在种子池中随机挑选12条分类指令和19条非分类指令，然后加上新生成的指令，按模版格式组织之后，输入给模型，让模型输出新生成的指令是否分类任务
  - 模板的格式如下：
  - 以下任务是否可以被视为具有有限输出标签的分类任务？
    - 任务1：根据我的性格和工作，告诉我是否适合。
    - 是否是分类任务？是
    - 任务2：举一个你必须使用幽默感的例子。
    - 是否是分类任务？否
    - 任务3：将给定文本中的占位符替换为适当的命名实体。
    - 是否是分类任务？否
    - 任务：（目标任务的说明）
    - 是否是分类任务？

Self-Instruct:生成任务输入和输出

- 在生成任务指令阶段，模型生成了任务的描述，但还缺乏相关任务的输入输出
- 本步骤的目标是为先前生成的指令配对输入和输出



Self-Instruct:生成任务输入和输出

- 采用上下文学习方法，通过使用其他任务的“指令-输入-输出”上下文示例作为提示，引导大语言模型为新任务生成输入和输出
- 给定指令之后，生成（输入, 输出）有两种策略。
  - 先生成输入，后生成输出，即输入优先策略
  - 生成输出，后生成输入，即输出优先策略

Self-Instruct:生成任务输入和输出

- 为什么会有输出优先策略?
  - 因为输入优先的方式在生成输入时，偏向于只生成一个标签，尤其是指令对应着分类任务时，其输入里面偏向于只生成一个类别。输出优先就是为了一定程度上缓解该问题
  - 首先生成所有可能的输出标签，再根据任务指令和输出补充相应的输入。

Self-Instruct:生成任务输入和输出

- 输入优先模板：

Come up with examples for the following tasks. Try to generate multiple examples when possible. If the task doesn't require additional input, you can generate the output directly.

Task: Sort the given list ascendingly.

Example 1

List: $[10,92,2.5,-4,92,5,101]$

Output: $[-4,2.5,5,10,92,92,101]$

Example 2

List: $[9.99,10,-5,-1000,5\ e6,999]$

Output: $[-1000,-5,9.99,10,999,5e6]$

Task: Converting $85\ \mathrm{F}$ to Celsius.

Output: $85°\mathrm{F} = 29.44°\mathrm{C}$

Task: {Instruction for the target task}

Self-Instruct:生成任务输入和输出

- 输出优先模板：

Given the classification task definition and the class labels, generate an input that corresponds to each of the class labels. If the task doesn't require input， just generate the correct class label.

Task: Classify the sentiment of the sentence into positive, negative, or mixed. Class label: mixed

Sentence: I enjoy the flavor of the restaurant but their service is too slow. Class label: Positive

Sentence: I had a great day today. The weather was beautiful and I spent time with friends. Class label: Negative

Sentence: I was really disappointed by the latest superhero movie. I would not recommend it.

Task: Tell me if the following email is a promotion email or not.

Class label: Promotion

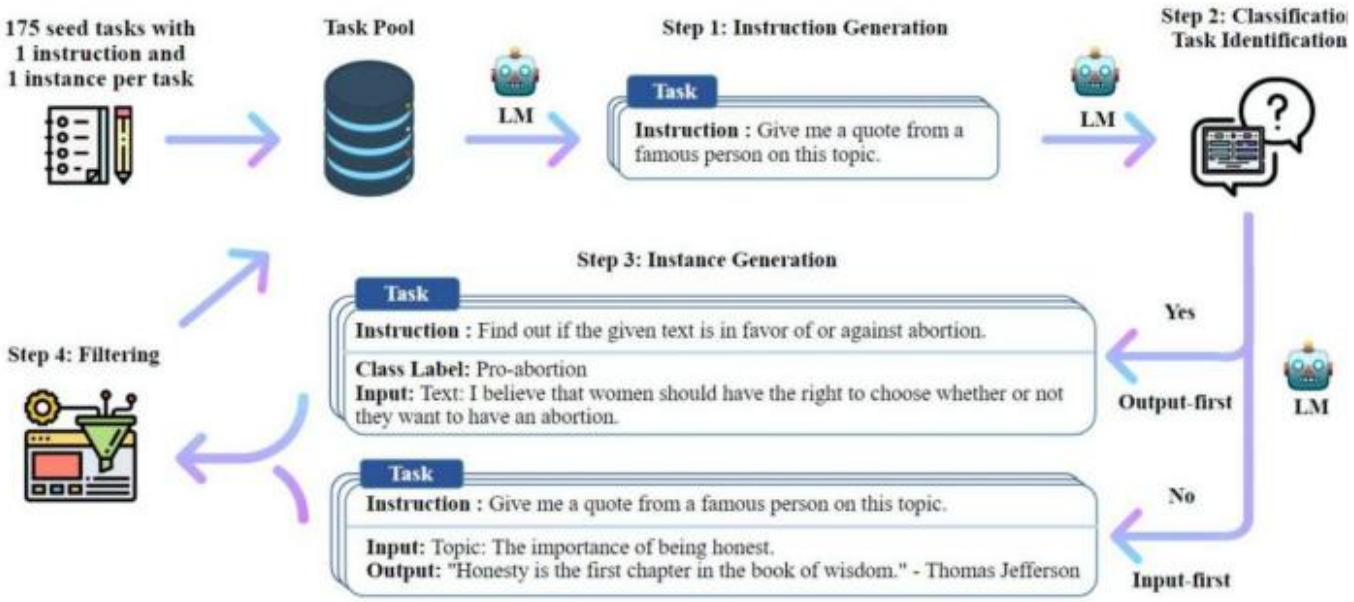
Email: Check out our amazing new sale! We've got discounts on all of your favorite products. Class label: Not Promotion

Email: We hope you are doing well. Let us know if you need any help.

Task: {Instruction for the target task}

Self-Instruct:过滤低质量数据

- 为了避免生成相关性过高的数据，需要对新生成的指令进行筛选



Self-Instruct:过滤低质量数据

- ROUGE-L筛选：只有与种子池中的任务 ROUGE-L 小于 0.7 时才会添加进入种子池
- 启发式规则进行筛选：
  - 删除特定关键词
  - 去除重复指令
  - 剔除过长或过短的指令

Self-Instruct:总结

- 使用Self-Instruct可以生成大量多样化且高质量的指令数据
- 在Llama模型上进行有监督微调，得到性能优异的Alpaca模型
- 数据示例：

Instruction:

Paraphrase the following sentence and keep the same meaning.

input: It is raining heavily outside.

output:

Rains are pouring down heavily outside.

开源数据集介绍

- 根据指令任务的类型，开源指令数据集可以大致分为两类：传统 NLP 任务指令和通用对话指令。

指令集名称	指令集大小	语言	构建方式	指令类型
Super-Natural Instructions	500 万	多语言	手动构建	NLP 任务指令
Flan2021	44 万	英语	手动构建	NLP 任务指令
pCLUE	120 万	中文	手动构建	NLP 任务指令
OpenAssistant Conversations	16.1 万	多语言	手动构建	通用对话指令
Dolly	1.5 万	英语	手动构建	通用对话指令
LIMA	1000	英语	手动构建	通用对话指令
Self-Instruct	5.2 万	英语	自动生成	通用对话指令
Alpaca data	5.2 万	英语	自动生成	通用对话指令
BELLE	150 万	中文	自动生成	通用对话指令

主要内容

- 提示学习与上下文学习
- 有监督精调的基本原理
- 指令数据构建
- 微调策略

全量微调

- 类似于传统的有监督学习，全量微调通过对模型的全部参数进行更新，来适应特定的任务或指令数据集

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} Loss(f_\theta(x_i), y_i)$$

- 通常使用交叉熵函数，对参数θ进行更新

全量微调的问题

- 计算开销大
- 对存储和训练时间要求较高
- 容易过拟合，导致泛化能力下降
- 在资源充足、目标任务重要性高、或者需要大幅调整模型行为时使用

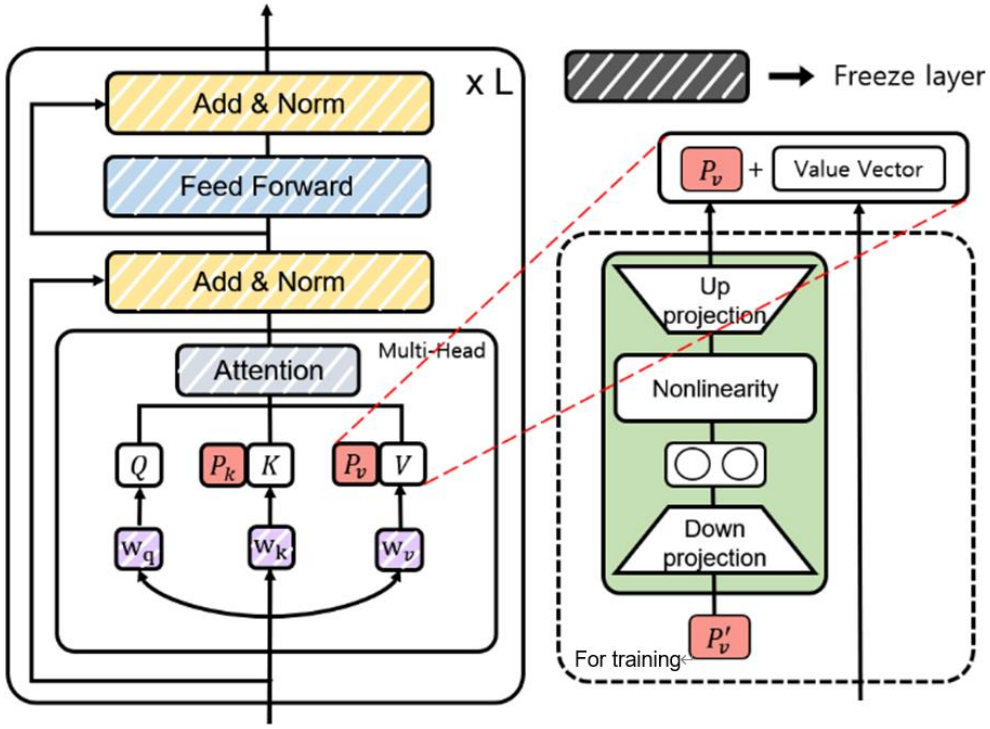
参数高效微调

- 参数高效微调（Parameter-Efficient Fine-Tuning，PEFT）是指在冻结模型大部分参数的前提下，仅调整少量参数，使模型能够高效适应下游任务
  - 前缀微调（Prefix Tuning）
  - 适配器（Adapter）方法
  - LoRA

参数高效微调

补充

- 前缀微调（Prefix Tuning）
  - 在输入序列前增加可训练的前缀向量，实现了对模型的轻量化调整



参数高效微调

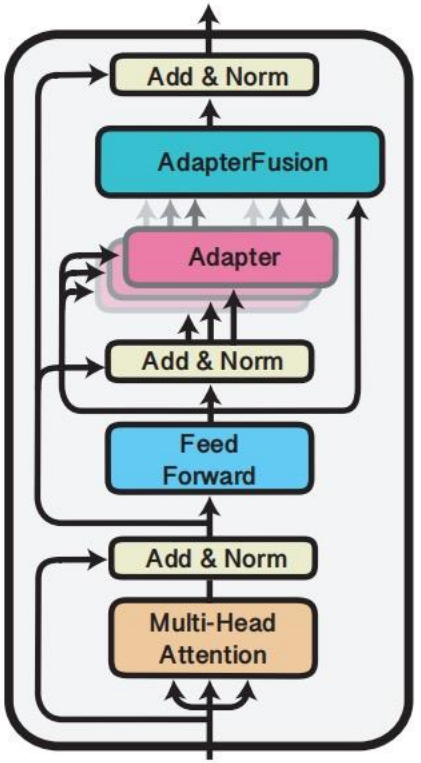
补充

- 前缀微调（Prefix Tuning）的核心机制
  - **引入前缀向量：**对于每一层，引入前缀向量。前缀向量不对应任何实际的词汇，而是作为虚拟的上下文向量存在
  - **调整自注意力**：在每一层，将前缀通过线性变换生成前缀的键和值，将这些前缀键值与原始输入的键值进行拼接计算注意力
  - **前缀向量的参数化**：每一层的前缀矩阵是一组可训练的参数，这些参数在微调过程中被优化。前缀向量的初始化通常采用随机初始化或基于预训练模型的嵌入初始化。

参数高效微调

补充

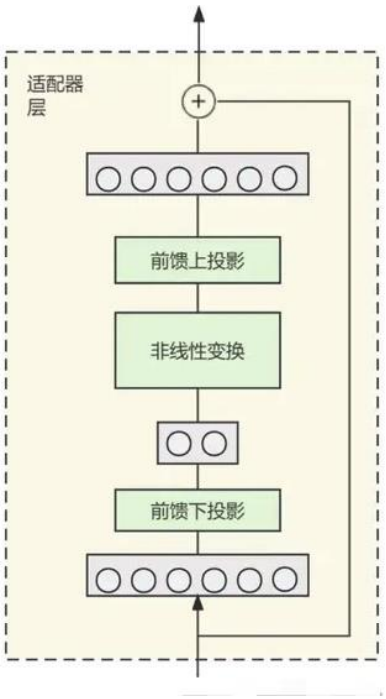
- 适配器（Adaptor）方法
  - 在模型的中间层插入小型可训练模块，兼顾灵活性和资源消耗
  - 微调时冻结预训练模型主体，由 Adapter 模块学习特定下游任务的知识



参数高效微调

补充

- 适配器（Adapter）方法
  - 每个 Adapter 模块由两个前馈子层组成：
    - 第一个前馈子层将 Transformer 块的输出作为输入，将原始输入维度 d（高维特征） 投影到 m（低维特征），通过控制 m 的大小来限制 Adapter 模块的参数量，通常情况下 m<<d
    - 通过第二个前馈子层还原输入维度，将 m （低维特征）重新投影到 d（原来的高维特征），作为 Adapter 模块的输出

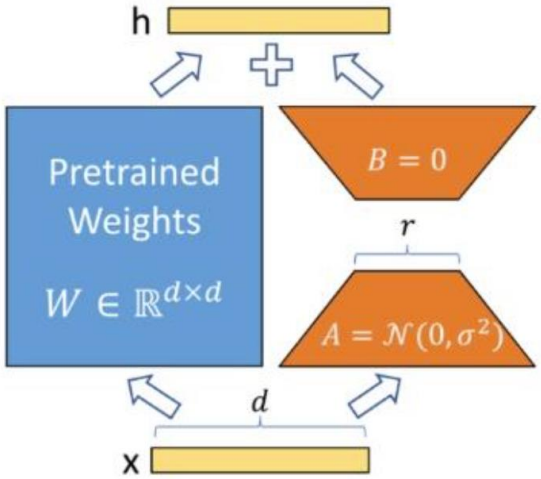


LoRA微调

- 假设：
  - 模型权重的更新可以用低秩矩阵表示
- 通过在原始权重的旁路添加低秩的可训练矩阵并冻结原始模型权重，大幅减少了需要训练的参数量和显存占用
- LoRA 不仅能有效降低资源需求，还在多个下游任务中表现出接近全量微调的性能，成为参数高效微调领域的重要方法之一

LoRA微调的基本原理

- 预训练权重为W_0,训练更新后参数为$W_1 = W_0 + \Delta W$
- 用低秩矩阵对 ΔW进行近似
- $\Delta W = \mathrm{BA}$
- 其中， 设$W \in R^{d\times k}$,则$B \in R^{d\times r}, A \in R^{r\times k}$



低秩分解的作用

补充

- 预训练权重为W_0,训练更新后参数为$W_1 = W_0 + \Delta W$
- 用低秩矩阵对 ΔW进行近似，$\Delta W = \mathrm{BA}$
- 其中，　设$W \in R^{d\times k}$,则$B \in R^{d\times r}, A \in R^{r\times k}$
- 设d=1000,k=1000,r=10,（r<<d）
  - 原本需要更新参数为1000*1000
  - 低秩分解后只需要更新1000*10+1000*10的参数

LoRA微调的技术细节

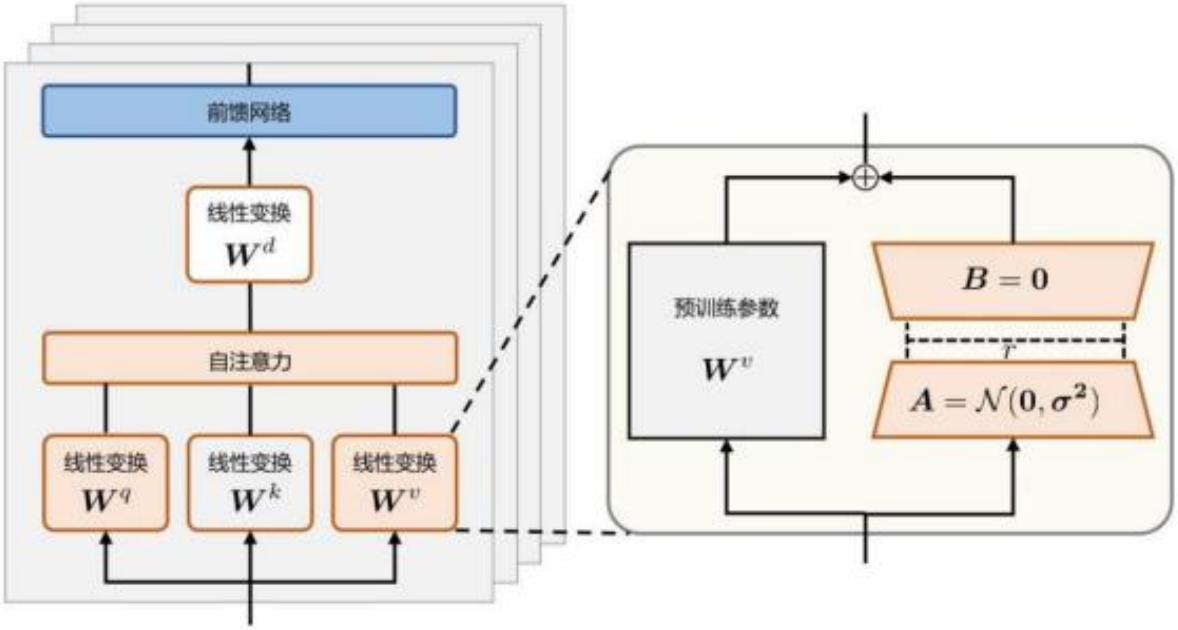
补充

- 秩的选择：
  - **r越大**：矩阵维度越高，训练参数也越多，模型对新数据的适应能力越强，但也增加了计算和内存的需求，可能导致过拟合
  - **r越小**：矩阵维度越小，训练参数也越小，减少了计算和内存的需求，但是可能会影响模型性能
  - 数据集大小、下游任务复杂性、基座模型能力？

LoRA微调的技术细节

补充

- 微调参数：
  - **LoRA方法可以应用于不同的模块：**一般只对每层的Self-Attention的部分做微调



PEFT的方法对比

特性	全参数微调	Prefix-tuning	Adapter	LoRA
训练参数量	全部	**0.1%-10%**	**2%-5%**	**0.1%-1%**
是否修改原参数	是	否	否	否
计算开销	高	中等	中等	低
推理速度影响	无	略增加	略增加	几乎无
性能	最佳	略低于全参数	略低于全参数	接近全参数微调

适配器方法会增加网络的深度，从而在模型推理时带来额外的时间开销，当没有使用模型并行或数据并行时，这种开销会较为明显

前缀微调方法会受到模型最大输入长度的影响，随着软提示的参数量增多，实际输入序列的最大长度也会相应减小，从而影响模型性能

LoRA变体：AdaLoRA

- LoRA秩的选择：
  - LoRA方法中不同层、不同模块的矩阵使用相同的秩r
  - 不同层和模块的权重对特定任务的重要性可能存在差异
- AdaLoRA (Adaptive Budget Allocation for PEFT)通过动态调整每个权重矩阵的秩大小来优化资源分配

LoRA变体：QLoRA

- QLoRA结合了参数量化技术，进一步压缩了模型的存储和计算需求，同时保持高效微调的能力
- QLoRA 在超大模型上表现尤为突出： 65B模型在单张48G GPU上微调并保持原本 16-bit 微调的性能。
- QLoRA的三个核心创新：
  - 4-bit NormalFloat (NF4) 数据类型
  - 双重量化 (Double Quantization)
  - 分页优化器 (Paged Optimizers)

特定功能的有监督精调

- **通用的训练——通用的能力**
- 模型在许多任务上展现了较强的能力，但在一些具体的能力上有所缺失
- 特定功能的有监督微调用于提升模型某方面具体的能力
  - 长上下文
  - 推理
  - 生成特定格式的输出
  - ……

课后习题

- 简述有监督精调（SFT）与提示学习（Prompt Learning）、上下文学习（In-context Learning）的关系。它们之间有何相似点和不同点？
- 为什么经过预训练的大语言模型需要通过有监督精调来提升其遵循指令的能力？微调过程中数据质量的重要性体现在哪些方面？
- 在设计一个用于有监督精调的数据集时，如何保证数据的多样性、准确性和覆盖面？请举例说明具体策略。
- 比较全量微调与参数高效微调（如 LoRA 和 Prefix Tuning）的优缺点，并说明在什么情况下更适合使用参数高效微调方法

课后习题

- 请举例说明有哪些特定功能的有监督精调？请分析它们在实际应用中的重要性。
- 当前长上下文处理的主流方法有哪些？结合实例分析这些方法的优势与局限性。

课后习题-答案

- 简述有监督精调（SFT）与提示学习（Prompt Learning）、上下文学习（In-context Learning）的关系。它们之间有何相似点和不同点?

有监督精调（SFT）、提示学习（Prompt Learning）和上下文学习（In-context Learning）是三种不同的学习方式，但它们有一些相似点。它们的共同点在于都不需要从头开始训练模型，而是利用预训练语言模型的知识来完成特定任务。

提示学习通过设计特定的输入提示来引导模型执行任务，不需要对模型进行参数调整。提示设计的关键是通过调整输入的语义，快速适应任务。

上下文学习则提供一些任务示例，使得模型能够根据这些示例推理并完成新任务，不同于提示学习的是它通过上下文中的示例进行任务推理，而不是仅仅依赖提示。

有监督精调则是在提示学习和上下文学习的基础上，通过对模型进行具体任务的有监督训练，进一步调整模型的参数，使其更好地执行任务。

它们之间的主要区别在于有监督精调需要更新模型的参数，而提示学习和上下文学习则不需要更新模型参数。

课后习题-答案

- 为什么经过预训练的大语言模型需要通过有监督精调来提升其遵循指令的能力？微调过程中数据质量的重要性体现在哪些方面？

预训练的大语言模型通过无监督学习获得了大量的语言知识，但它们缺乏明确的任务执行能力，尤其是在执行具体指令时。有监督精调通过引入标注数据，使模型能够更好地理解任务并按照指令生成准确的响应。

微调过程中数据质量非常重要，主要体现在：

数据多样性：确保数据能够覆盖各种任务类型和情境，避免模型只学会应对特定的情况。

数据准确性：高质量的数据确保模型学习到正确的任务指令和对应的输出。

数据覆盖面：广泛的覆盖面可以确保模型能够适应多样化的任务和复杂的场景。

课后习题-答案

- 在设计一个用于有监督精调的数据集时，如何保证数据的多样性、准确性和覆盖面？请举例说明具体策略。

在设计用于有监督精调的数据集时，可以采用以下策略来确保数据的多样性、准确性和覆盖面：

多样性：通过采集来自不同领域、不同任务类型的数据，确保数据涵盖不同的任务场景和输入输出对。例如，LIMA数据集通过采样收集数据，保证了任务的多样性。

准确性：确保数据标注的质量，避免低质量的标注影响模型性能。例如，手动构建的指令数据集经过筛选和人工标注，确保高质量。

覆盖面：包括各种任务和问题，确保数据集能够代表目标应用场景的广泛任务。

课后习题-答案

- 比较全量微调与参数高效微调（如 LoRA 和 Prefix Tuning）的优缺点，并说明在什么情况下更适合使用参数高效微调方法

全量微调通过更新所有参数来适应特定任务，能够灵活地调整模型并适应不同的任务，但计算开销大，易于过拟合，需要大量的计算资源。

参数高效微调（如LoRA和Prefix Tuning）：这类方法通过冻结大部分模型参数，仅微调少量参数，显著降低了计算开销，并且可以在保持高效性能的同时节省资源。LoRA特别适用于多个下游任务，减少了微调所需的参数数量。

适合使用参数高效微调方法的场景是当计算资源有限，需要对多个下游任务进行微调时，能够节省存储和计算资源。

课后习题-答案

- 请举例说明有哪些特定功能的有监督精调？请分析它们在实际应用中的重要性。

特定功能的有监督精调：例如，数字推理、代码生成与理解、多轮对话等。

数字推理：增强模型在数学计算和符号推理方面的能力。

代码生成：使得模型能够理解并生成符合要求的代码。

多轮对话：提升模型在多轮对话中的上下文保持能力。

这些特定功能的有监督精调对于提升模型在复杂任务中的表现具有重要意义，特别是在需要精确推理或生成特定格式输出的场景中。

课后习题-答案

- 当前长上下文处理的主流方法有哪些？结合实例分析这些方法的优势与局限性。

当前长上下文处理的主流方法包括：

增加上下文窗口：通过扩展模型的输入长度，适应更长的文本。

位置编码方法：调整模型的编码方式，使其能够处理更长的上下文。

插值法：对长文本进行拆分并重新组合。

这些方法的优势在于能够提升模型处理长文本的能力，但它们也存在一定的局限性，例如增加上下文窗口可能导致训练效率低下，位置编码方法可能面临效率和复杂度的挑战。